



Institut Spécialisé de Technologie Appliquée de Mirleft

Manipuler les systèmes de contrôle de version (Git/Github)

M. Bentaleb Saad

Plan :

- **C'est quoi un SCV ?**
- **C'est quoi Git ?**
- **Comment fonctionne Git ?**
- **Commandes de Git (Local Repository)**
- **Manipuler les branche en Git**
- **GitHub/GitLab**
- **Commandes de Git (Remote Repository)**

C'est quoi un SCV (système de contrôle de version) ?

Qu'est-ce qu'un système de contrôle de version ?

Un **système de contrôle de version** (SCV) est un outil qui permet de **gérer les modifications** apportées à des fichiers et des documents **au fil du temps**. Il est particulièrement utilisé dans le développement de logiciels, mais peut également être appliqué à d'autres types de projets.



Fonctionnalités principales :

Historique des modifications : Permet de garder une **trace** de toutes les **modifications** apportées aux fichiers, y compris **qui a fait les changements et quand**.

Collaboration : Facilite le **travail en équipe** en permettant à plusieurs personnes de travailler sur le même projet **sans écraser les modifications des autres**.

Restauration : Permet de **revenir à des versions antérieures** des fichiers en cas d'erreurs ou de problèmes.



Types de systèmes de contrôle de version :

Systèmes locaux :

- Gèrent les versions sur un seul ordinateur (ex : RCS).

Systèmes centralisés :

- Utilisent un serveur central pour stocker les versions (ex : Subversion).

Systèmes distribués :

- Chaque utilisateur a une copie complète du dépôt, ce qui permet de travailler hors ligne (ex : Git).

C'est Quoi Git ?



Définition

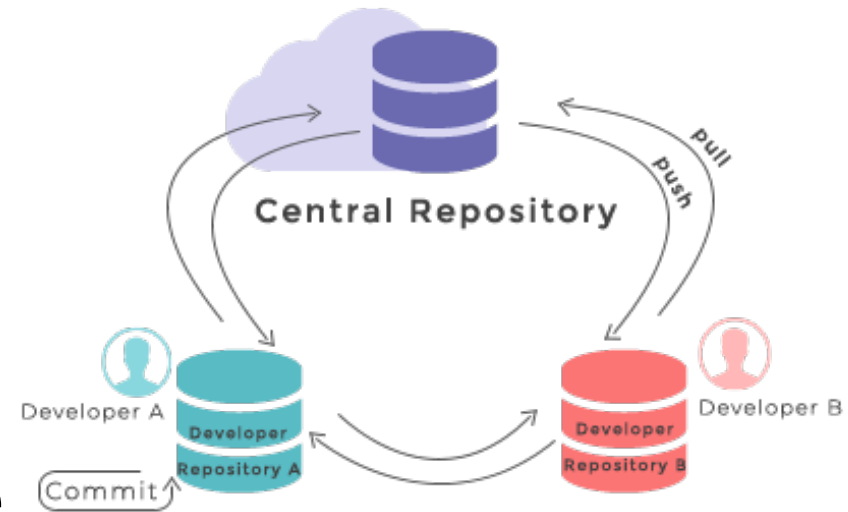
Git est un **système de gestion de versions** distribué qui permet de suivre les modifications de fichiers et de faciliter le travail collaboratif, en offrant un historique complet et une gestion avancée des branches.

Comment Fonctionne Git ?



Un Dépôt Git (Repository)

Est un **répertoire** (base de données) de projet qui **stocke tous les fichiers**, la documentation et le code source, ainsi que **l'historique complet des modifications** de chaque fichier.

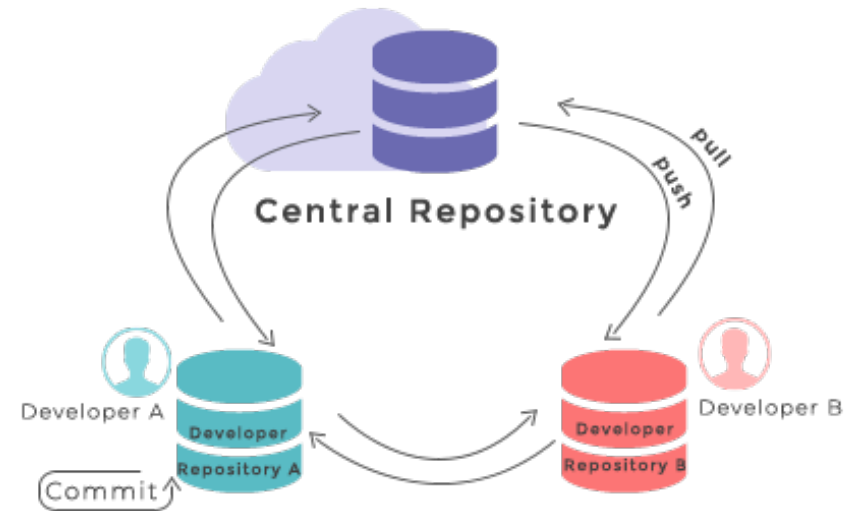


Creation d'une Repository

git init: Lorsque vous exécutez cette commande dans un répertoire, Git initialise un nouveau dépôt Git local dans ce répertoire.

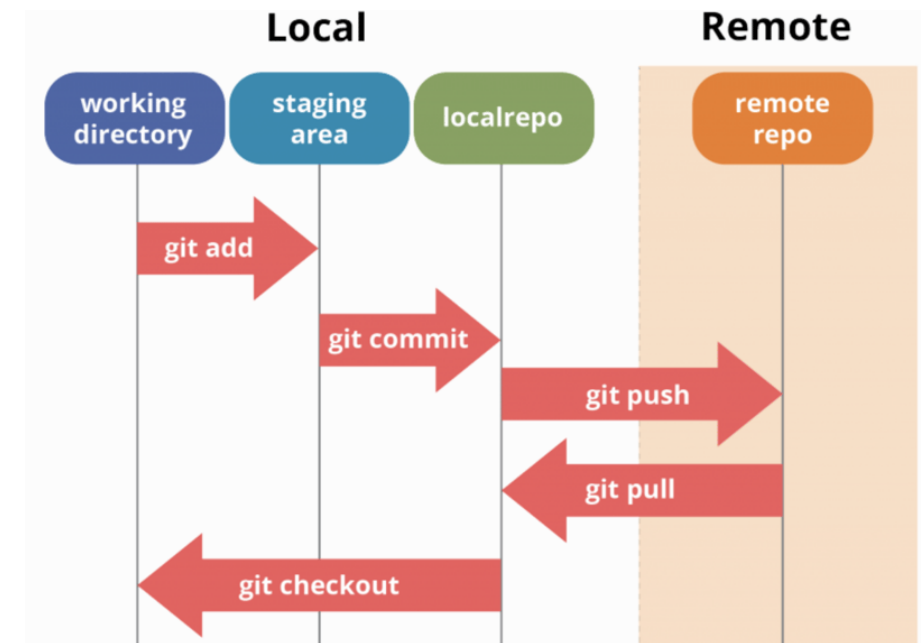
Un dossier caché nommé `.git` est créé dans le répertoire actuel.

C'est le dossier que Git utilise pour suivre gérer le controle de version.



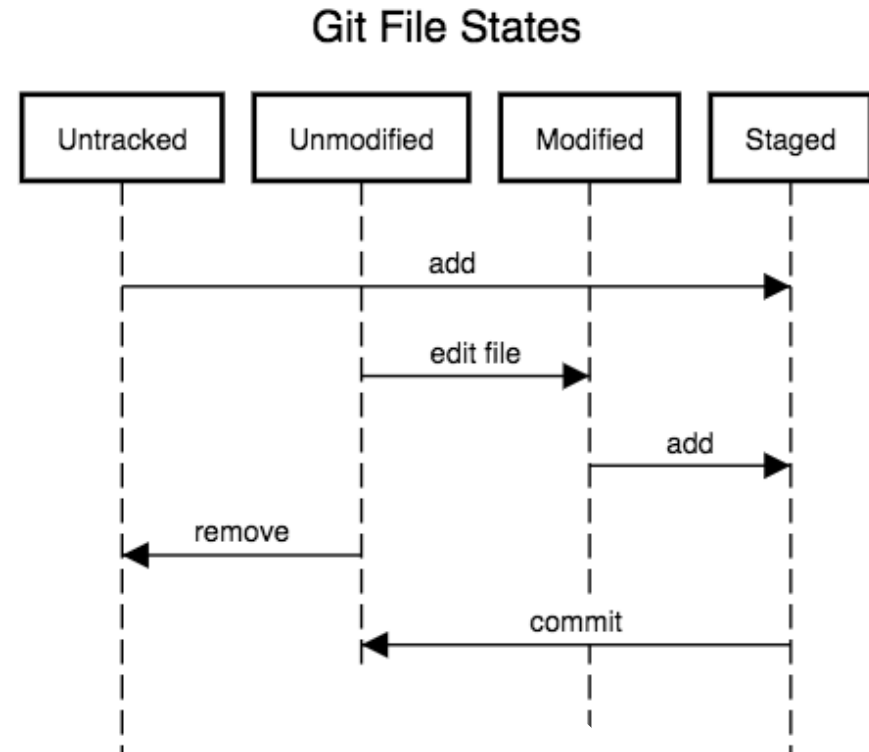
Le Workflow de Git

- **Working Directory:** Le dossier local où se trouvent les fichiers de votre projet. Ce sont les fichiers que vous modifiez directement.
- **Staging Area:** Une zone temporaire où les modifications sont préparées avant d'être validées (commit) dans le dépôt.
- **Local Repo:** Le dossier ".git" dans votre projet qui contient l'historique complet des modifications et des commits du dépôt.
- **Remote Repo:** Une version de votre dépôt hébergée sur un serveur ou une plateforme cloud (ex. : GitHub, GitLab, Bitbucket).



Les États d'un fichier en Git

- **Untracked:** Le fichier existe dans (Working directory) mais n'est pas encore suivi par Git.
- **Unmodified:** Le fichier est suivi par Git et correspond à la version du dernier commit.
- **Modified:** Le fichier est suivi par Git, mais des modifications ont été apportées depuis le dernier commit.
- **Staged(préparé):** Le fichier est suivi et les modifications ont été ajoutées pour le prochain commit.



Commandes de Git (Local Repository)



Statut d'un dépôt

- **git status:** Affiche le statut actuel de repository avec des informations détaillées.
- **git status -s:** Affiche en bref le statut de repository

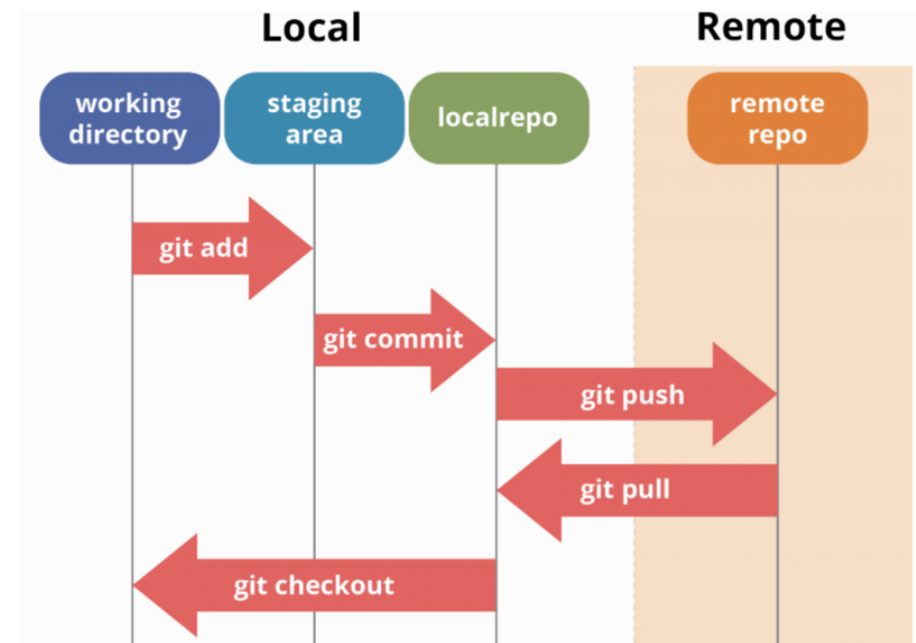
```
HiManshu@HiManshu-PC MINGW64 ~/Desktop/NewDirectory (master)
$ touch demofile

HiManshu@HiManshu-PC MINGW64 ~/Desktop/NewDirectory (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        demofile

nothing added to commit but untracked files present (use "git add" to track)
```

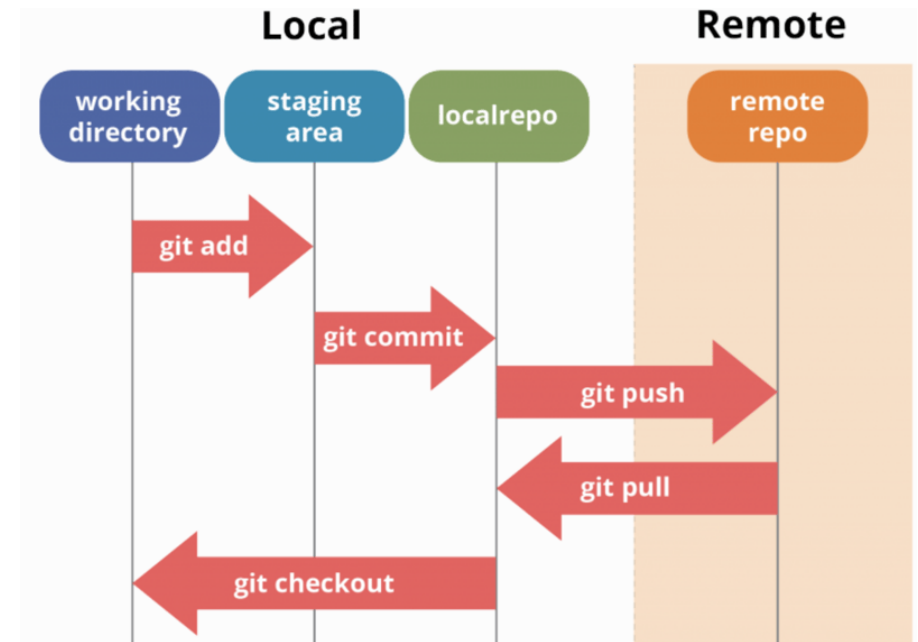
Commandes de preparation (Staging)

- **git add fileName:** transférer le fichier non suivi du “Working Directory” vers “Staging Area”.
- **git add . :** transférer tous les fichiers non suivi vers “Staging Area”.
- **git restore --staged (filename)/(.):** Récupérer les fichiers de “Staging Area”



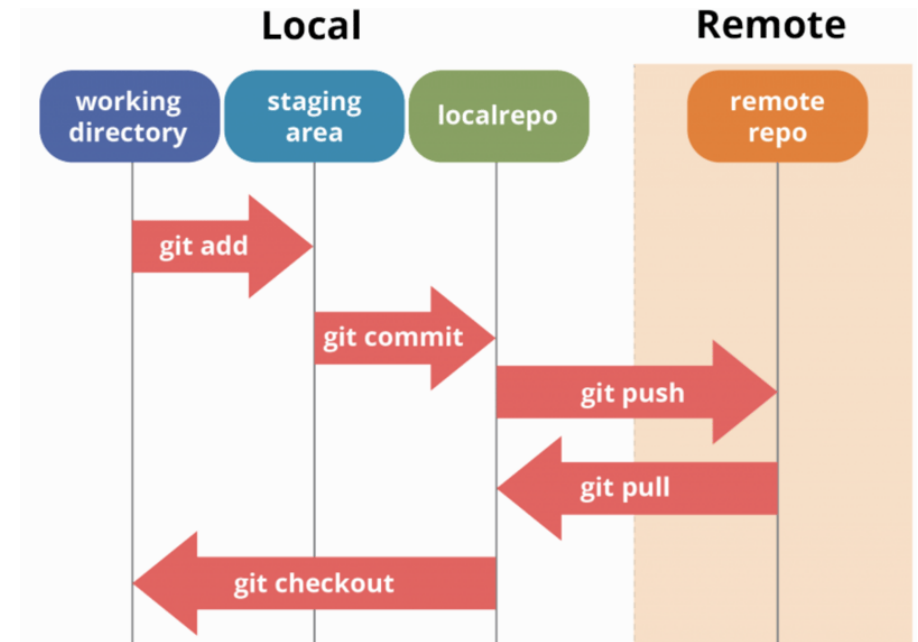
Commande de validation (Committing)

- **git commit -m**
“message”: valider les modifications et les transférer de « Staging Areas » vers « Local Repository »



Nettoyage des dépôts

- **git clean -n:** Afficher une liste des fichiers non préparés (Not staged).
- **git clean -f:** Supprimer tous les fichiers non préparés



Configurations en Git

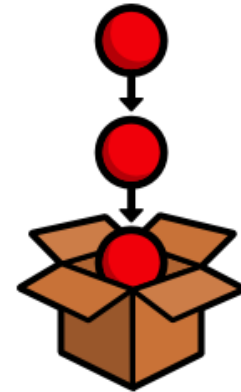
- **git config [--global] -l**: Afficher la liste des configurations en Git.
- **git config [--global] configName**: Afficher la valeur d'une config spécifique.
- **git config [--global] configName value**: Affecter une valeur à une config spécifique.
- **Git config [--global] --unset configName**: Supprimer un config spécifique.
- **Remarque**: Avec le mot **--global** on peut spécifier si la config va être appliquée sur le dépôt uniquement ou sur tout le système git



git config
management

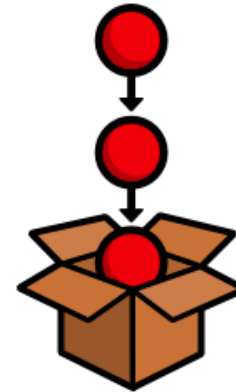
Stashing

Dans Git, **Stashing** (stockage) est un moyen de sauvegarder temporairement les modifications sans les valider (**Commit**). C'est utile lorsque vous souhaitez changer de branche ou travailler sur autre chose, mais que vous n'êtes pas prêt à valider vos modifications actuelles. Stashing enregistre votre travail et laisse votre répertoire de travail (**Workign Directory**) propre, ce qui vous permet de revenir et d'appliquer les modifications ultérieurement.



Les commandes de stashing

- **git stash:** Stocker tous les fichiers non validé (**Uncommitted**) dans un stash
- **git stash list:** Afficher la liste de tous les stashes.
- **git stash pop:** Extraire les fichiers du stash récente.
- **git stash show stash@{ID}:** Afficher details d'un stash spécifique.
- **git stash pop stash@{ID}:** Extraire les fichiers d'un stash spécifique.
- **git stash drop stash@{ID}:** Supprimer un stash spécifique.
- **git stash clear:** Supprimer tous les stash.



Ignorer le suivi d'un fichier/dossier

Pour ignorer le suivi d'un fichier ou dossier par , on crée un fichier “**.gitignore**” qui indique à Git quels fichiers ou répertoires ignorer et ne pas suivre dans le dépôt. Il est utilisé pour éviter de suivre des fichiers inutiles ou sensibles.

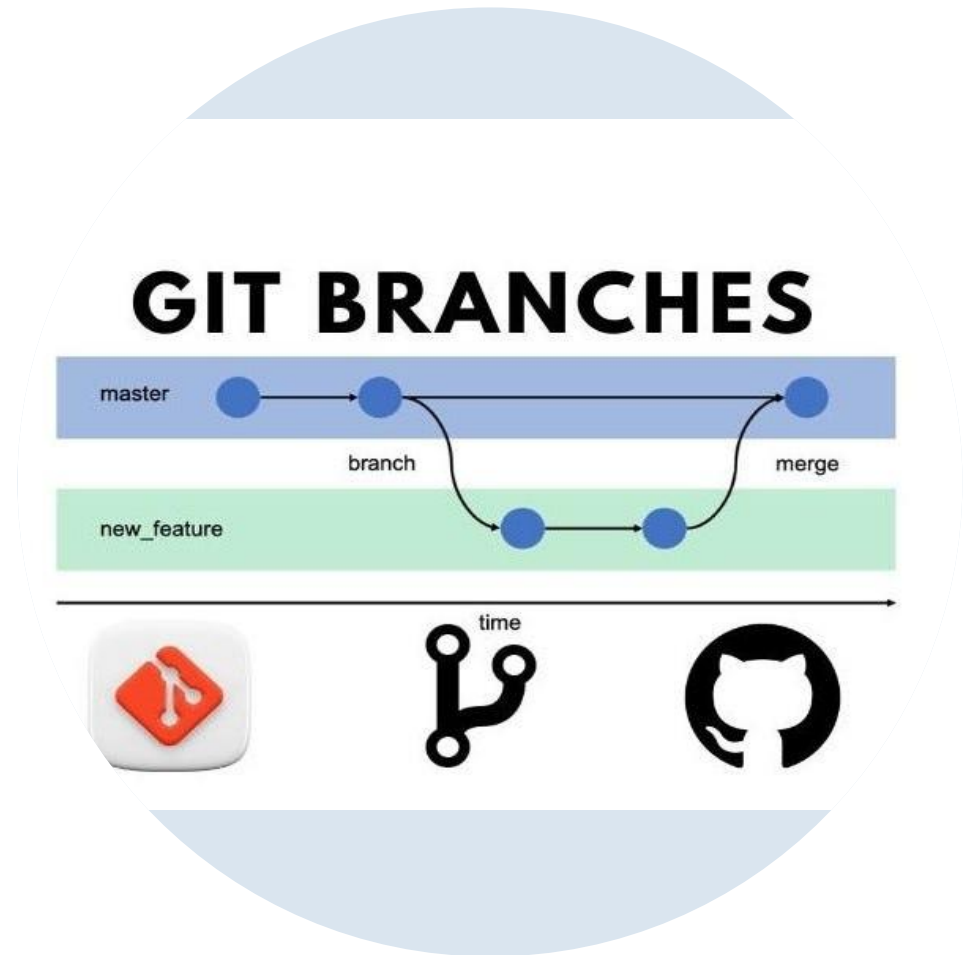
- **Remarque:** Seuls les fichiers non suivis seront ignorés. Pour ignorer un fichier déjà suivi on utilise la commande suivante: **git rm --cached fileName** puis on l'ajoute dans le fichier **.gitignore** et ensuite on valide les changement (commit).



Pour mieux comprendre le contenu du fichier .gitignore

```
# Example of what a ".gitignore" file contains:  
# *.log      => Ignore all the files with ".log" extension  
# !vip.log   => Ignore all files with ".log" extension except the file "vip.log"  
# index.html => Ignore every "index.html" file in the project  
# node_packs/ => Ignore every "node_packs" directory in the project
```

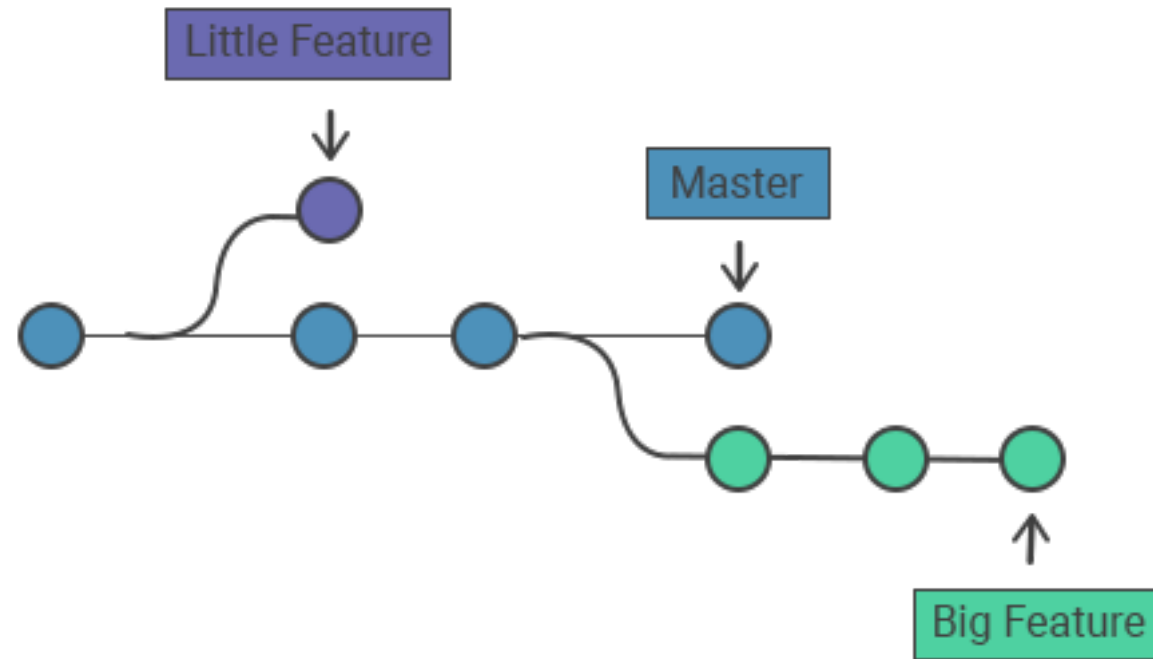
Manipuler Les Branches En Git



C'est Quoi le branching

Le branching Git permet de travailler sur différentes versions d'un projet en même temps. Vous pouvez créer des branches pour de nouvelles fonctionnalités, des corrections de bugs ou des expérimentations sans affecter le code principal.

Example

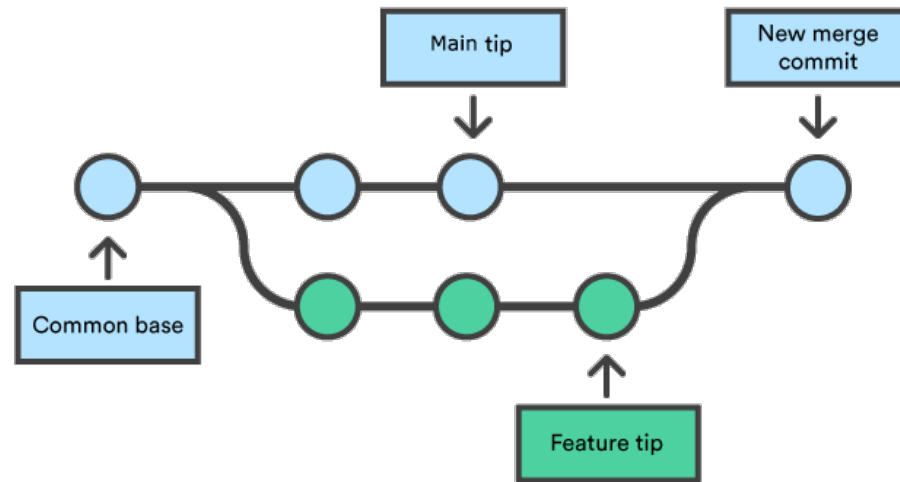


Commandes de Branching

- **git branch:** Afficher la liste des branches
- **git branch branchName:** Créer une branche.
- **git branch checkout branchName:** Changer la branche vers une autre branche.
- **git branch -m newName:** Renommer une branche.
- **git branch -D branchName:** Supprimer une branche spécifique.

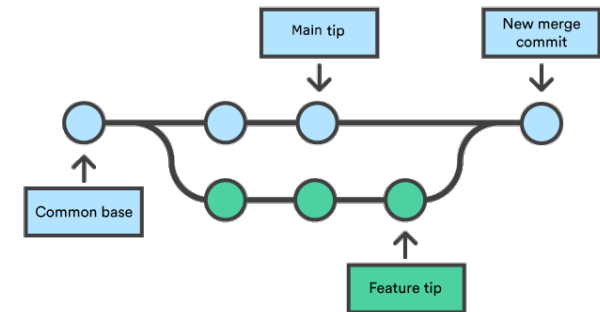


Branches Merging



Fusion de branches (Branches Merging)

- **git merge branchName:**
Fusionner la branche spécifié avec la branche actuelle.
- **Remarque 1:** En cas d'absence de conflits, les modifications sont directement validées.
- **Remarque 2:** En cas de conflits, nous devons les résoudre puis valider les modifications.



Historique de branche (Log)

- **git log:** Afficher l'historique de tous les commits (Chaque commit a son propre HASH).
- **git checkout HASH:** Charger le code d'une version (Commit) spécifique.

